ECE551: Digital System Design and Synthesis

Fall 2026

Comments

Coding Style & Effect on Synthesis

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Comments

- Comments are VERY important!
  - In industry, others will have to read your code
  - Some poor soul (perhaps you 4 years later) are going to have to reference your code when a customer discovers a bug

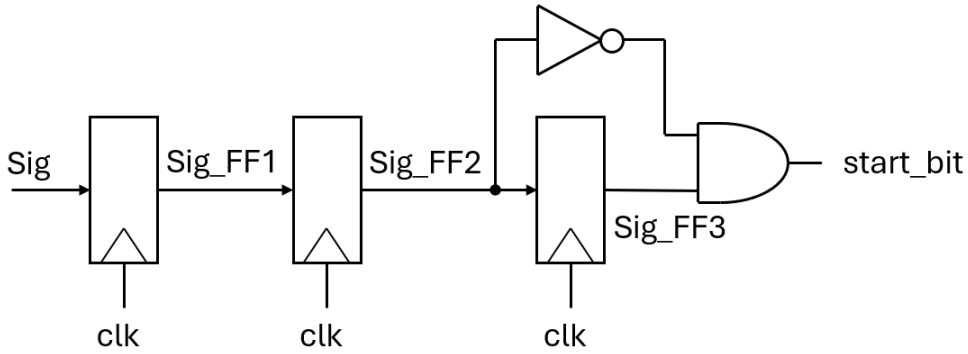
- The best comments document **why** you are doing what you are doing
  - Motivation, thought process

- Good to have module-level or block comments
  - Avoids having to read detailed Verilog code if just browsing through

Comments Example

```
always_ff @(posedge clk)
begin
   Sig_FF1 <= Sig;       // Capture value of Sig Line in FF
   Sig_FF2 <= Sig_FF1;   // Flop Sig_FF1 to form Sig_FF2
   Sig_FF3 <= Sig_FF2;   // Flop Sig_FF2 to form Sig_FF3
end

// start_bit is ~Sig_FF2 & Sig_FF3
assign start_bit = (~Sig_FF2 && Sig_FF3) ? 1'b1 : 1'b0;
```

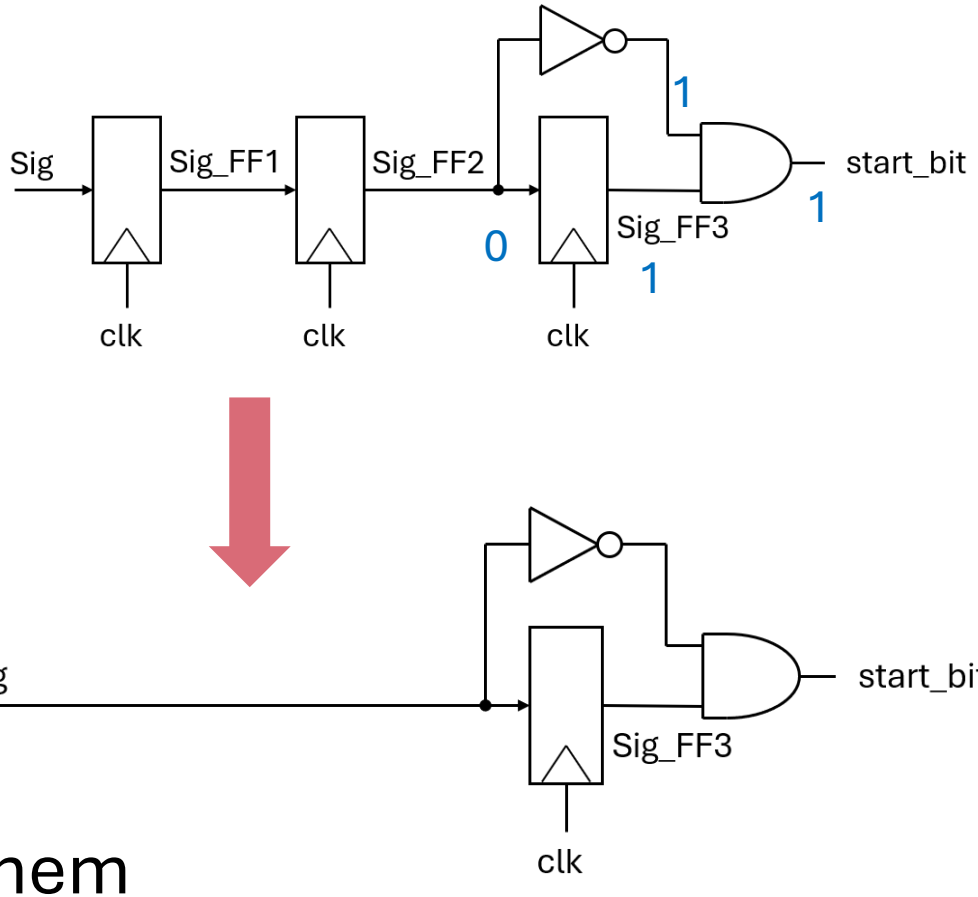


- Comments don't say more than what Verilog code says…

Comments Example

```
always_ff @(posedge clk)
begin
   Sig_FF1 <= Sig;       // Capture value of Sig Line in FF
   Sig_FF2 <= Sig_FF1;   // Flop Sig_FF1 to form Sig_FF2
   Sig_FF3 <= Sig_FF2;   // Flop Sig_FF2 to form Sig_FF3
end

// start_bit is ~Sig_FF2 & Sig_FF3
assign start_bit = (~Sig_FF2 && Sig_FF3) ? 1'b1 : 1'b0;
```



- Optimize away unused logic!
- Save on area and power!

  ...Right??

... turns out we can't! there was a reason for them

Comments Example

```
always_ff @(posedge clk)
/*************************************************
* Sig is asynchronous and has to be double flopped for *
* metastability reasons prior to use                   *
*************************************************/
begin
   Sig_FF1 <= Sig;
   Sig_FF2 <= Sig_FF1;  // double flopped metastability free
   Sig_FF3 <= Sig_FF2;  // flop again for use in edge detection
end

/*************************************************
* Start bit in protocol initiated by falling edge of   *
* Sig line                                             *
*************************************************/
assign start_bit = (~Sig_FF2 && Sig_FF3) ? 1'b1 : 1'b0;
```

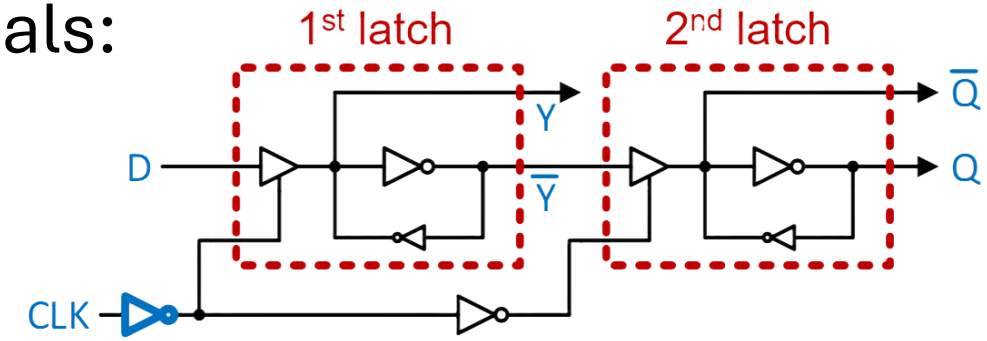
Multi-line comment starts with /* and ends with */

Comment to end of line is //

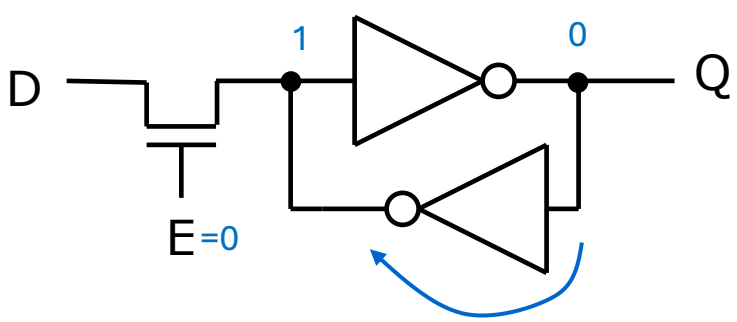
- This is better commenting. It tells you why stuff was done.

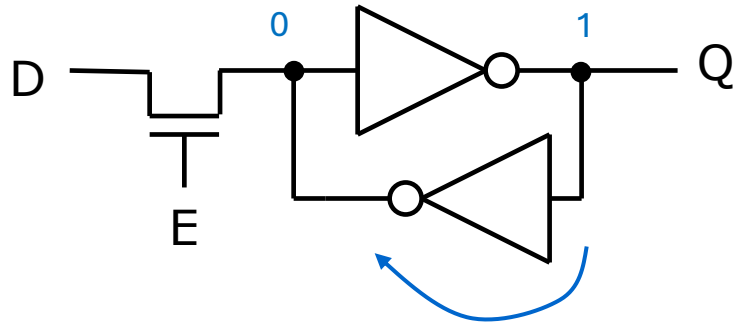
Aside: Metastability

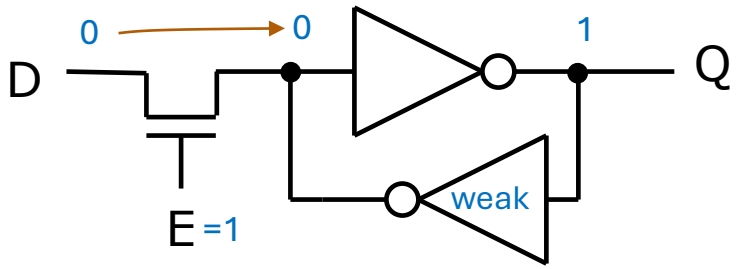
- FF may go metastable if input changes as FF is trying to capture value
- FF internals:



- Latch operation:

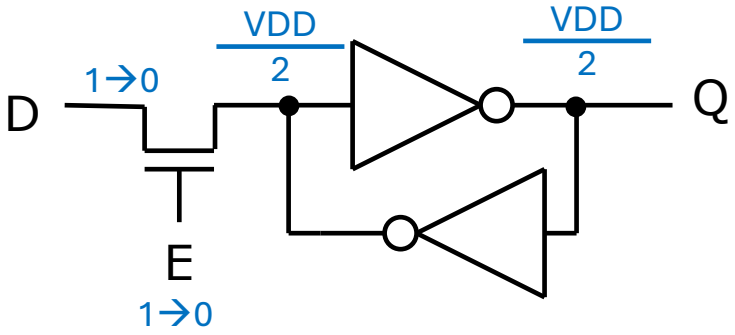


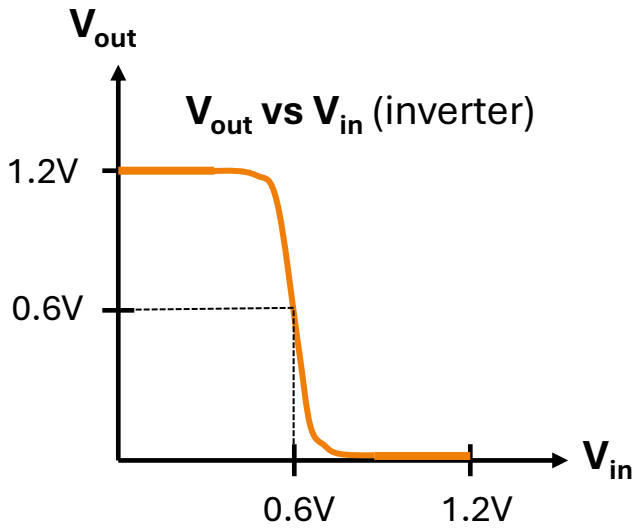


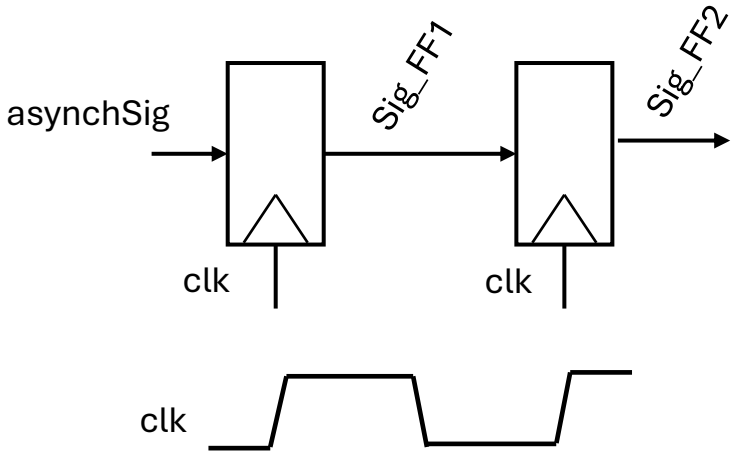


Aside: Metastability

- FF may go metastable if input changes as FF is trying to capture value
- If D input change value during setup window:







The first flop can go metastable…
But we give it a whole clock period to resolve.

Takeaway: Always double flop asynch signals before using them

Good HDL “Self Comments”

- Use descriptive module names, signal names

```
module xyz123 (A, B) ;
  input [3:0] A ;
  output B ;

  //module contents
endmodule
```

vs

```
module doomsday_machine (
  input [3:0] crystals,
  output earthquake
);
  //module contents
endmodule
```

- Naming conventions:
  - Use mixed case and/or _ to delimit descriptive names

```
assign parityErr = ^serial_reg;
nxtState = returnRegister;
```

  - Consistent suffix for signals of same type
    - Use **_n** for active low

```
rst_n = 1’b0; // assert reset
```

Implicit net declarations

- Always declare all signals before use
- When signal is used but not declared, it will be **implied**
  - Default to single bit wire…

```
module majority(output out, input a, b, c);
  assign part1 = a & b;
  assign part2 = a & c;
  assign part3 = b & c;
  assign out = part1 | part2 | part3;
endmodule

module fulladder(output Sum, Cout, input A, B, Cin);
  assign {Cout, Sum} = A + B + cin;
endmodule
```

- Disable by using `` `default_nettype none `` directive

Parameters

- Make code more generic / flexible
  - Defined locally to the module
  - Can be overridden (passed a value in an instantiation)

```
module adder(a,b,cin,sum,cout);
  parameter WIDTH = 8; // default is 8
  input [WIDTH-1:0] a,b;
  input cin;
  output [WIDTH-1:0] sum;
  output cout;

  assign {cout,sum} =  a  +  b + cin;

endmodule
```

```
module alu(src1,src2,dst,cin,cout);
  input [15:0] src1,src2;
  ...
  ////////////////////////////////
  // Instantiate 16-bit adder //
  ////////////////////////////////
  adder #(16) add16(.a(src1),.b(src2),.cin(cin),
                    .cout(cout),.sum(dst));
  ...
endmodule
```

Parameters

- Make code more generic / flexible
  - Defined locally to the module
  - Can be overridden (passed a value in an instantiation)

```
module paramReg
  #(parameter WIDTH = 8, CLK2Q = 0)
(
  input [WIDTH-1:0] D,
  input clk,
  input rst_n,
  output [WIDTH-1:0] Q
);
  always @(posedge clk, negedge rst_n)
    if (!rst_n)
      Q <= {WIDTH{1'b0}};
    else
      Q <= #CLK2Q D;
endmodule
```

ANSI declaration style

```
// Instantiate 16-bit reg
paramReg #(.WIDTH(16),.CLK2Q(1)) iDUT(.clk(clk), .rst_n(rst_n),
                                      .D(Din), .Q(Qmon));
```

Pass params by name is better than by order

localparam

- Like parameter, but cannot be overridden
- Useful for constants in the code that might be subject to change

```
localparam P_COEFF = 4'h6;        // P_term coefficient
localparam D_COEFF = 7'h38;       // D_term coefficient
```

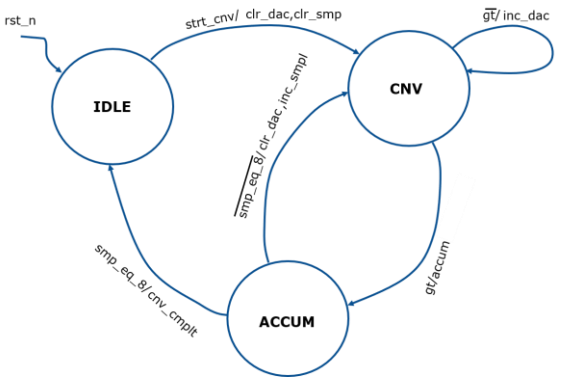
Enumerated Type

- System Verilog feature

{IDLE=2'b00,CONV=2'b10,ACCM=2'b11}

```
typedef enum logic [1:0] { IDLE, CONV, ACCM } state_t;

state_t state, nxt_state;     // declare state and nxt_state signals
```




```
// use enum when referring to state
case (state)
  IDLE : if (strt_cnv) begin
         clr_dac = 1;
         clr_smpl = 1;
         nxt_state = CONV;
  end
```

```
localparam IDLE = 2'b00;
localparam CONV = 2'b01;
localparam ACCM = 2'b10;
```



vs



waveform shows enums

waveform shows encoding…

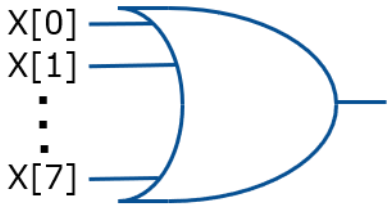
Can Coding Style Affect Synthesis?

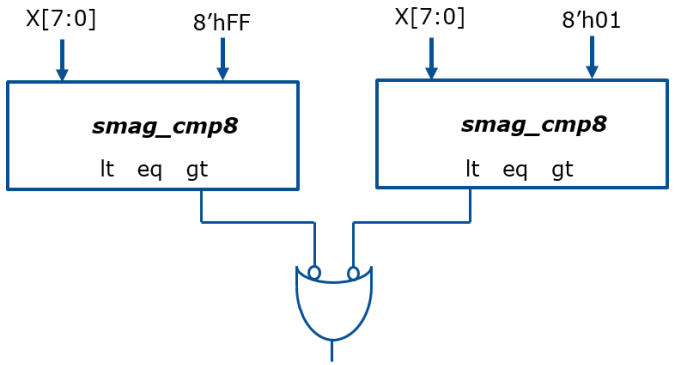
- HDLs abstract hardware design → do we have to consider the hardware?
  - Good hardware design requires ability to analyze a problem to find simplifications
  - Multiple variables: throughput, area, latency, power
  - Finding an optimal hardware implementation is a computationally complex problem → synthesis tool need guidance on where to start

```
logic [7:0] X;
if (X!=0)     vs     if ( (X<=-1) || (X>=1) )
```

What hardware might this generate?

If any bit of x is 1, then it is not 0





Latches with Continuous Assign

```
assign q_out = enable? data_in : q_out;
```

- What does this do?
  - It acts like a latch
  - It simulates like a latch

- How does it synthesize?

